Sorting I
Bubble Sort, Selection Sort, Insertion Sort

Data Structures & Object-Oriented Programming



Table of Contents

- Big-O Notation
- Bubble Sort
- Selection Sort
- Insertion Sort
- Code ของทั้ง 3 อัลกอริทึม

ปัญหาของการค้นหาข้อมูลที่ไม่เรียงลำดับ

จินตนาการสถานการณ์ต่อไปนี้

- มหาวิทยาลัยแห่งหนึ่งมีรายชื่อนิสิตทั้งหมด 10,000 คน เก็บไว้ในลำดับที่ไม่ได้เรียงตามตัวอักษรใดๆ เลย (เช่น เรียงตามลำดับการสมัครเข้าเรียน)
- หากต้องการค้นหาว่านิสิตชื่อหนึ่งอยู่ในรายชื่อนี้หรือไม่ วิธีเดียวที่ทำได้คือการค้นหาแบบ linear search คือไล่ตรวจดูทีละรายชื่อตั้งแต่ต้นจนกว่าจะพบ หรือจนกว่าจะตรวจครบทุกคนแล้วไม่พบเลย
- หากคำนวณโดยเฉลี่ย การค้นหาแบบนี้ต้องตรวจสอบประมาณครึ่งหนึ่งของรายชื่อทั้งหมด (5,000 คน) ต่อการค้นหาหนึ่งครั้ง
- ในกรณีที่เลวร้ายที่สุด (ชื่อที่ค้นหาอยู่ท้ายสุด หรือไม่มีอยู่ในรายชื่อเลย) จำเป็นต้องตรวจสอบครบทั้ง 10,000 คน ความซับซ้อนของการค้นหาแบบนี้จึงเป็น O(n)

Binary Search

- แต่หากรายชื่อนิสิตทั้ง 10,000 คนนี้ถูกเรียงตามลำดับตัวอักษรไว้ล่วงหน้าแล้ว เราสามารถใช้เทคนิคที่เรียกว่า **binary search** แทนได้
- หลักการของ binary search คือ
  - เปิดดูรายชื่อที่ตำแหน่งกึ่งกลางก่อน
  - หากชื่อที่ต้องการค้นหาเรียงอยู่ก่อนชื่อกึ่งกลางตามลำดับตัวอักษร ก็สามารถตัดทิ้งครึ่งหลังทั้งหมดได้ทันที โดยไม่ต้องตรวจสอบเลยแม้แต่รายชื่อเดียว เพราะรู้แน่นอนว่าชื่อที่ต้องการไม่มีทางอยู่ในครึ่งนั้น
  - จากนั้นจึงทำซ้ำขั้นตอนเดิมกับครึ่งที่เหลืออยู่ โดยเปิดกึ่งกลางของครึ่งนั้นอีกครั้ง ตัดทิ้งอีกครึ่งหนึ่ง
  - ทำเช่นนี้ไปเรื่อยๆ จนกว่าจะพบ

Binary Search (2)

- ประสิทธิภาพของ binary search
  - สำหรับรายชื่อ 10,000 คน จะใช้เพียง 14 ขั้นตอน เท่านั้น (คำนวณจาก $\log_2 10{,}000 \approx 13.29$ ปัดขึ้นเป็น 14) เปรียบเทียบกับ linear search ที่อาจต้องตรวจสอบมากถึง 10,000 ขั้นตอนในกรณีเลวร้ายที่สุด
  - ความแตกต่างนี้มีนัยสำคัญอย่างยิ่งเมื่อข้อมูลมีขนาดใหญ่ขึ้น
- อย่างไรก็ตาม binary search มีเงื่อนไขบังคับที่ขาดไม่ได้เลย นั่นคือ ข้อมูลที่จะค้นหาต้องเรียงลำดับแล้วเท่านั้น
- หากข้อมูลไม่ได้เรียงลำดับไว้ล่วงหน้า หลักการ “ตัดทิ้งครึ่งหนึ่ง” จะใช้ไม่ได้เลย เพราะไม่มีทางรู้ได้ว่าชื่อที่ต้องการอยู่ฝั่งไหนของจุดกึ่งกลาง
- ข้อสรุปที่สำคัญที่สุดจากประเด็นนี้คือ การ sort ข้อมูลให้เรียงลำดับ คือเงื่อนไขเบื้องต้น (prerequisite) ที่ทำให้ algorithm ที่มีประสิทธิภาพสูงอย่าง binary search สามารถทำงานได้เลย
- การลงทุนเวลาในการ sort ข้อมูลไว้ล่วงหน้าครั้งเดียว จึงมักคุ้มค่ากว่าการค้นหาแบบ linear search ซ้ำแล้วซ้ำเล่าในระยะยาว

O(n log n) เทียบกับ O(n²)

- ในทางทฤษฎีว่า O(n²) นั้น “ช้ากว่า” O(n log n)
- คำถาม คือ ทั้งสองอัลกอริทึมนี้ต่างกันมากเพียงใดเมื่อข้อมูลมีขนาดใหญ่ขึ้น
- ตารางต่อไปนี้ คำนวณจำนวน operation และเวลาโดยประมาณ (สมมติว่าคอมพิวเตอร์ทำงานได้ 10^9 operations ต่อวินาที ซึ่งใกล้เคียงกับความเร็วของ CPU ทั่วไปในปัจจุบัน) สำหรับข้อมูลจำนวน 1,000,000 รายการ

Complexity	จำนวน Operations	เวลาโดยประมาณ
O(n log n)	ประมาณ 20,000,000	ประมาณ 0.02 วินาที
O(n²)	1,000,000,000,000 (หนึ่งล้านล้าน)	ประมาณ 17 นาที

ตัวเลขในตารางคำนวณจากสูตรตรงๆ: $n \log_2 n \approx 1{,}000{,}000 \times 19.93 \approx 19{,}930{,}000$ ปัดเป็นประมาณ 20 ล้าน และ $n^2 = 1{,}000{,}000^2 = 10^{12}$
เมื่อหารด้วยความเร็ว 10^9 operations/วินาที จะได้ 0.02 วินาที และ 1,000 วินาที ($\approx$ 16.7 นาที) ตามลำดับ)

O(n log n) เทียบกับ O(n²)

- ความแตกต่างระหว่างสองแนวทางนี้คือ 50,000 เท่า พอดี (คำนวณจาก 1,000,000,000,000 ÷ 20,000,000)
- ตัวเลขนี้ หมายความว่างานเดียวกันที่ algorithm หนึ่งทำเสร็จในเวลา (0.02 วินาที) อีก algorithm หนึ่งอาจต้องใช้เวลานานพอ (17 นาที)
- นี่คือเหตุผลที่แท้จริงว่าทำไมนักพัฒนาซอฟต์แวร์จึงให้ความสำคัญกับการเลือก algorithm ที่มีความซับซ้อนต่ำ
- ไม่ใช่เพราะความสวยงามทางทฤษฎี แต่เพราะผลกระทบที่จับต้องได้จริงต่อประสบการณ์ผู้ใช้งาน

สิ่งที่จะได้เรียนในสัปดาห์นี้

- sorting algorithms สามตัวแรก ได้แก่ Bubble Sort, Selection Sort และ Insertion Sort
- ทั้งสามตัวมีความซับซ้อนแบบ $O(n^2)$ เหมือนกันในกรณีทั่วไป
- ถึงแม้จะมีความซับซ้อนแบบเดียวกันในภาพรวม แต่ละอัลกอริทึมก็มีจุดแข็งและพฤติกรรมที่แตกต่างกันอย่างมีนัยสำคัญ
- ในสัปดาห์หน้า เราจะได้เรียนรู้ Merge Sort และ Quick Sort ซึ่งเป็นอัลกอริทึมที่มีความซับซ้อนแบบ O(n log n) ซึ่งเป็นกลุ่มที่แสดงให้เห็นถึงประสิทธิภาพที่เหนือกว่าอย่างมาก
  - ทั้งสองอัลกอริทึมใช้แนวคิดที่เรียกว่า Divide and Conquer คือ การแบ่งปัญหาขนาดใหญ่ออกเป็นปัญหาย่อยที่เล็กลง แก้ปัญหาย่อยแต่ละส่วนแยกกัน แล้วจึงนำผลลัพธ์มารวมกลับเข้าด้วยกัน
  - แนวคิดแตกต่างจาก 3 อัลกอริทึมที่จะเรียนในสัปดาห์นี้ (ทำงานกับข้อมูลทั้งก้อนไปพร้อมกันตลอดเวลา)
- การเรียนรู้ $O(n^2)$ sorting algorithms ก่อนในสัปดาห์นี้จึงเป็นพื้นฐานสำคัญที่ทำให้เข้าใจแนวคิดของการเปรียบเทียบและการจัดเรียงข้อมูลแล้ว
- ในบางสถานการณ์เฉพาะ (เช่นข้อมูลขนาดเล็ก หรือข้อมูลที่เกือบเรียงอยู่แล้ว) อัลกอริทึมกลุ่ม $O(n^2)$ เหล่านี้กลับทำงานได้เร็วกว่า O(n log n) ซึ่งเป็นประเด็นที่หลายคนมักไม่ทราบ

Big-O Notation

Big-O บอกอัตราการเติบโต ไม่ใช่เวลาที่แน่นอน

- Big-O ไม่ได้บอกเวลาที่แน่นอนในการทำงาน แต่บอก อัตราการเติบโต (rate of growth) ของเวลาที่ใช้เมื่อขนาดของข้อมูล (n) เพิ่มขึ้น
- เช่น เมื่อกล่าวว่า algorithm หนึ่งมีความซับซ้อนเป็น $O(n^2)$ ไม่ได้หมายความว่า algorithm นั้นจะใช้เวลา n^2 วินาที แต่หมายความว่า
  - หากขนาดของข้อมูลเพิ่มขึ้นเป็น 2 เท่า เวลาที่ใช้จะเพิ่มขึ้นเป็นประมาณ 4 เท่า (เพราะ $(2n)^2 = 4n^2$)
  - หากขนาดข้อมูลเพิ่มขึ้นเป็น 10 เท่า เวลาที่ใช้จะเพิ่มขึ้นเป็นประมาณ 100 เท่า (เพราะ $(10n)^2 = 100n^2$)
  - นี่คือความหมายที่แท้จริงของคำว่า “อัตราการเติบโต”

กฎการอ่าน Big-O จากโค้ด

- วิธีที่ตรงไปตรงมาที่สุดในการประเมิน Big-O ของโค้ดคือการนับจำนวน loop ที่ซ้อนกันและพิจารณาว่าแต่ละ loop มีขอบเขตขึ้นอยู่กับ n หรือไม่

```
# O(n²) — inner loop ขึ้นอยู่กับ n และซ้อนอยู่ภายใน outer loop ที่ขึ้นอยู่กับ n เช่นกัน
for i in range(n):
    for j in range(n):
        do_something()

# O(n) — มี loop เดียวที่ขึ้นอยู่กับ n
for i in range(n):
    do_something()

# O(1) — ไม่มี loop ใดขึ้นอยู่กับ n เลย การเข้าถึง index ใช้เวลาคงที่เสมอ
x = arr[0]
```

- หาก loop สองชั้นซ้อนกันและทั้งคู่ขึ้นอยู่กับ n จะได้ $O(n^2)$
- หาก loop สามชั้นซ้อนกันจะได้ $O(n^3)$
- และเป็นเช่นนี้ไปเรื่อยๆ ตามจำนวนชั้นของ loop ที่ซ้อนกัน
- หาก loop สองชั้นไม่ได้ซ้อนกันแต่วางเรียงต่อกัน (loop แรกทำงานเสร็จแล้วจึงเริ่ม loop ที่สอง) ความซับซ้อนโดยรวมจะเป็นเพียง $O(n)$ ไม่ใช่ $O(n^2)$ เพราะ Big-O สนใจเฉพาะพฤติกรรมเมื่อ n มีขนาดใหญ่มาก และ $O(n) + O(n)$ ยังคงเป็น $O(n)$ เพียงแต่มีค่าคงที่คูณอยู่ (ซึ่ง Big-O จะละทิ้งค่าคงที่นี้ไปเสมอ)

Best, Average, Worst Case

- Algorithm 1 ตัวสามารถมีพฤติกรรมที่แตกต่างกันได้อย่างมาก ขึ้นอยู่กับลักษณะของ input ที่ได้รับ
- ด้วยเหตุนี้เราจึงมักพิจารณา algorithm ผ่านสามมุมมองต่อไปนี้เสมอ
  - **Best case** คือ สถานการณ์ที่ input มีลักษณะที่เอื้อประโยชน์ต่อ algorithm มากที่สุด
    - ทำให้ algorithm ทำงานเสร็จเร็วที่สุดเท่าที่จะเป็นไปได้
    - เช่น สำหรับ sorting algorithm หลายตัว input ที่เรียงลำดับไว้อยู่แล้วมักเป็น best case
  - **Worst case** คือ สถานการณ์ที่ input มีลักษณะที่เลวร้ายที่สุดต่อ algorithm
    - ทำให้ algorithm ต้องทำงานนานที่สุดเท่าที่จะเป็นไปได้
    - เช่น สำหรับ sorting algorithm หลายตัว input ที่เรียงลำดับกลับหัวโดยสมบูรณ์ (มากไปน้อยเมื่อต้องการเรียงน้อยไปมาก) มักเป็น worst case
  - **Average case** คือ สถานการณ์ที่ input มีลักษณะสุ่มทั่วไป ไม่ได้เอื้อประโยชน์หรือเป็นโทษต่อ algorithm เป็นพิเศษ
    - ในทางคณิตศาสตร์การคำนวณ average case มักซับซ้อนกว่าสองกรณีแรกมาก เพราะต้องพิจารณาความน่าจะเป็นของ input ทุกรูปแบบที่เป็นไปได้

Best, Average, Worst Case (2)

- เมื่อมีการกล่าวถึงความซับซ้อนของ algorithm โดยไม่ได้ระบุกรณีให้ชัดเจน
  - เช่น เพียงแค่บอกว่า “algorithm นี้เป็น $O(n^2)$”
- โดยธรรมเนียมปฏิบัติทั่วไปมักหมายถึง Worst case เป็นค่า default เสมอ
- เหตุผลคือ worst case เป็นตัวรับประกันขอบเขตบนของเวลาที่ใช้ ซึ่งมีประโยชน์ในทางวิศวกรรมมากกว่า เพราะทำให้สามารถวางแผนและรับประกันประสิทธิภาพขั้นต่ำของระบบได้อย่างมั่นใจ ไม่ว่าจะเจอ input แบบใดก็ตาม

Bubble Sort

หลักการทำงานของ Bubble Sort

- Bubble Sort ทำงานโดยเปรียบเทียบ element ที่อยู่ติดกันเป็นคู่ๆ ไล่จากต้น array ไปจนถึงปลาย
- หากพบว่า element คู่ใดอยู่ผิดลำดับ (ตัวหน้ามากกว่าตัวหลัง เมื่อต้องการเรียงจากน้อยไปมาก) จะทำการสลับตำแหน่งทันที แล้วเดินหน้าต่อไปเปรียบเทียบคู่ถัดไป
- การไล่ตรวจครบหนึ่งรอบตั้งแต่ต้นจนจบ array เรียกว่าหนึ่ง pass
- ผลลัพธ์ที่น่าสนใจของกระบวนการนี้คือ
  - ทุกครั้งที่ทำ pass หนึ่งรอบจบ element ที่มีค่ามากที่สุดในส่วนที่ยังไม่เรียงจะถูก “ดัน” ไปอยู่ที่ตำแหน่งสุดท้ายของส่วนนั้นเสมอ
  - เปรียบได้กับฟองอากาศในน้ำที่ค่อยๆ ลอยตัวขึ้นสู่ผิวน้ำทีละน้อยในแต่ละครั้งที่มีการกวน
  - นี่คือที่มาของชื่อ “Bubble Sort”

Trace การทำงานบน [5, 3, 8, 1, 2]

- ลองไล่ดูการทำงานทีละขั้นตอนบน array ตัวอย่าง [5, 3, 8, 1, 2]
- Pass 1: เปรียบเทียบทีละคู่จากซ้ายไปขวา

```
[5,3] → 5>3 → swap → [3,5,8,1,2]
[5,8] → 5<8 → ok    → [3,5,8,1,2]
[8,1] → 8>1 → swap → [3,5,1,8,2]
[8,2] → 8>2 → swap → [3,5,1,2,8]   ← 8 (ค่ามากสุด) ถูกดันไปอยู่ปลายสุดแล้ว
```

- Pass 2: เนื่องจากตำแหน่งสุดท้ายเรียงถูกต้องแล้ว (คือเลข 8) ไม่จำเป็นต้องตรวจซ้ำอีก

```
[3,5] → 3<5 → ok    → [3,5,1,2,8]
[5,1] → 5>1 → swap → [3,1,5,2,8]
[5,2] → 5>2 → swap → [3,1,2,5,8]   ← 5 ถูกดันไปอยู่ตำแหน่งที่ 4 แล้ว
```

- Pass 3:

```
[3,1] → 3>1 → swap → [1,3,2,5,8]
[3,2] → 3>2 → swap → [1,2,3,5,8]   ← 3 อยู่ตำแหน่งที่ถูกต้องแล้ว
```

- Pass 4: ไล่ตรวจทั้งคู่ที่เหลือ ([1,2]) แต่ไม่มีการสลับเกิดขึ้นเลยแม้แต่คู่เดียว แสดงว่า array เรียงลำดับสมบูรณ์แล้ว จึงหยุดการทำงานทันที
- ผลลัพธ์สุดท้าย: [1, 2, 3, 5, 8] ✓

การปรับปรุงด้วย Early Termination

- เมื่อ pass ใดก็ตามไม่มีการสลับตำแหน่งเกิดขึ้นเลยแม้แต่คู่เดียว นั่นเป็นสัญญาณที่ชัดเจนว่า array ได้เรียงลำดับสมบูรณ์แล้ว ไม่จำเป็นต้องทำ pass ต่อไปอีกเลย
- เทคนิคการตรวจสอบเงื่อนไขนี้แล้วหยุดการทำงานทันทีเรียกว่า **early termination**
- ประโยชน์ของ early termination แสดงให้เห็นชัดเจนที่สุดเมื่อ array ที่ได้รับมาเรียงลำดับไว้อยู่แล้วตั้งแต่ต้น
  - ในกรณีนี้ Bubble Sort จะทำเพียง pass เดียว
  - ตรวจพบว่าไม่มีการสลับใดๆ เกิดขึ้นเลย แล้วหยุดการทำงานทันที
  - ทำให้ความซับซ้อนของ best case กลายเป็น $O(n)$ แทนที่จะเป็น $O(n^2)$ เหมือนกรณีทั่วไป

เหตุใด Inner Loop ต้องลดขนาดลงทุก Pass

- จากการ trace สังเกตได้ว่า
  - Pass 1 ทำการเปรียบเทียบทั้งหมด 4 คู่ (สำหรับ array ขนาด 5)
  - แต่ Pass 2 ทำการเปรียบเทียบเพียง 3 คู่เท่านั้น ไม่ใช่ 4 คู่เหมือนเดิม
- เหตุผลคือ หลังจบ pass แรก element ที่มีค่ามากที่สุดได้ถูกดันไปอยู่ที่ตำแหน่งสุดท้ายเรียบร้อยแล้ว
  - จึงไม่มีความจำเป็นใดๆ ที่จะต้องนำตำแหน่งนั้นมาเปรียบเทียบซ้ำอีกในรอบถัดไป เพราะรู้แน่นอนแล้วว่าตำแหน่งนั้นถูกต้องแล้ว
- ในเชิงโค้ด สิ่งนี้หมายความว่า ขอบเขตของ inner loop ต้องลดลงตามค่าของ pass ที่กำลังทำงานอยู่เสมอ
  - เขียนเป็นสูตรได้ว่า range(n - 1 - i)
    - โดยที่ i คือหมายเลข pass ปัจจุบัน (เริ่มนับจาก 0)
  - ไม่ใช่ range(n - 1) แบบตายตัวคงที่ทุก pass
  - หากเขียนโค้ดผิดโดยใช้ range(n - 1) ตายตัว จะทำให้เกิดการเปรียบเทียบตำแหน่งที่เรียงถูกต้องแล้วซ้ำโดยไม่จำเป็น
    - แม้จะไม่ทำให้ผลลัพธ์ผิดพลาด แต่ก็เป็นการเสียเวลาโดยเปล่าประโยชน์

Stability ของ Bubble Sort

- Bubble Sort เป็น stable sort กล่าวคือ element ที่มีค่าเท่ากันจะยังคงอยู่ในลำดับสัมพัทธ์เดิมหลังการเรียงลำดับเสมอ
- เหตุผลที่ Bubble Sort มีคุณสมบัตินี้อยู่ที่เงื่อนไขการสลับตำแหน่งที่ใช้ arr[j] > arr[j+1] (มากกว่าอย่างเคร่งครัด ไม่ใช่มากกว่าหรือเท่ากับ)
- เมื่อ element สองตัวมีค่าเท่ากันพอดี เงื่อนไขนี้จะเป็นเท็จ จึงไม่มีการสลับตำแหน่งเกิดขึ้น
    - ทำให้ลำดับสัมพัทธ์ระหว่าง element ที่เท่ากันไม่เปลี่ยนแปลงไปจากเดิม

Selection Sort

หลักการทำงานของ Selection Sort

- Selection Sort จะแบ่ง array ออกเป็นสองส่วนในทางความคิดเสมอ คือ
  - ส่วนซ้ายที่เรียงลำดับเสร็จแล้ว และ ส่วนขวาที่ยังไม่ได้เรียง
- ในแต่ละ pass, algorithm จะค้นหา element ที่มีค่าน้อยที่สุดในส่วนขวาที่ยังไม่เรียงทั้งหมด แล้วนำ element นั้นมาสลับตำแหน่งกับ element ตัวแรกของส่วนที่ยังไม่เรียง
- เป็นการขยายส่วนที่เรียงแล้วออกไปทีละหนึ่งตำแหน่งในแต่ละ pass

Trace การทำงานบน [5, 3, 8, 1, 2]

- i=0: ส่วนที่ยังไม่เรียงคือทั้งหมด [5,3,8,1,2]
  - ค้นหาค่าน้อยที่สุดพบว่าเป็น 1 ที่ตำแหน่ง index 3 จึงสลับตำแหน่ง 0 กับตำแหน่ง 3
  - swap(0, 3) → [1, 3, 8, 5, 2]
- **i=1:** ส่วนที่ยังไม่เรียงคือ [3,8,5,2] (ตำแหน่ง 1 ถึง 4)
  - ค้นหาค่าน้อยที่สุดพบว่าเป็น 2 ที่ตำแหน่ง index 4 จึงสลับตำแหน่ง 1 กับตำแหน่ง 4
  - swap(1, 4) → [1, 2, 8, 5, 3]
- **i=2:** ส่วนที่ยังไม่เรียงคือ [8,5,3]
  - ค้นหาค่าน้อยที่สุดพบว่าเป็น 3 ที่ตำแหน่ง index 4 จึงสลับตำแหน่ง 2 กับตำแหน่ง 4
  - swap(2, 4) → [1, 2, 3, 5, 8]
- **i=3:** ส่วนที่ยังไม่เรียงคือ [5,8]
  - ค้นหาค่าน้อยที่สุดพบว่าเป็น 5 ซึ่งอยู่ที่ตำแหน่ง index 3 อยู่แล้ว (ตรงกับตำแหน่งที่ควรอยู่พอดี) จึง**ไม่ต้องสลับ**
  - [1, 2, 3, 5, 8] (ไม่เปลี่ยนแปลง)
- **ผลลัพธ์สุดท้าย:** [1, 2, 3, 5, 8] ✓

Selection Sort เป็น $O(n^2)$ ทุกกรณี ไม่มีข้อยกเว้น

- Selection Sort ไม่มีทางเป็น O(n) ในกรณีใดๆ เลย แม้แต่ในกรณีที่ array เรียงลำดับไว้อยู่แล้วตั้งแต่ต้น
- เหตุผลอยู่ที่ธรรมชาติของ inner loop ซึ่งมีหน้าที่ ค้นหาค่าน้อยที่สุดในส่วนที่ยังไม่เรียง
- ไม่ว่า array เดิมจะเรียงลำดับมาอย่างไร inner loop จำเป็นต้องตรวจสอบทุก element ที่เหลืออยู่ให้ครบถ้วนเสมอ เพื่อให้มั่นใจได้ว่าไม่พลาดค่าที่น้อยที่สุดจริงๆ ไปแม้แต่ตัวเดียว
- ไม่มีทางที่จะ “early terminate” การค้นหาแบบนี้ได้เลย ไม่ว่าจะพบ element ที่น้อยกว่าตัวปัจจุบันหรือไม่ในระหว่างทาง ก็ยังต้องตรวจสอบทุกตัวที่เหลือต่อไปจนครบอยู่ดี
- ด้วยเหตุนี้ Selection Sort จึงมี Best, Average และ Worst case เป็น $O(n^2)$ เหมือนกันทั้งหมด ไม่มีข้อยกเว้นใดๆ
  - ต่างจาก Bubble Sort ที่ยังพอมี best case พิเศษจาก early termination

จุดแข็งเพียงหนึ่งเดียวของ Selection Sort

- Selection Sort มีจุดแข็งที่โดดเด่นมากอยู่หนึ่งประการ
  - จำนวนครั้งของการสลับตำแหน่ง (swap) น้อยที่สุดเท่าที่จะเป็นไปได้
  - จากตัวอย่างจะเห็นได้ว่า Selection Sort ทำการ swap สูงสุดเพียง n-1 ครั้ง ตลอดทั้ง algorithm เท่านั้น
    - ในตัวอย่างที่มี n=5 ทำ swap เพียง 3 ครั้ง เพราะ i=3 ไม่จำเป็นต้อง swap
    - เปรียบเทียบกับ Bubble Sort ที่อาจต้อง swap มากถึง $O(n^2)$ ครั้งในกรณีเลวร้ายที่สุด
- คุณสมบัตินี้มีประโยชน์อย่างมากในสถานการณ์ที่ การเขียนข้อมูล (write) มีต้นทุนสูงกว่าการอ่านข้อมูล (read) อย่างมีนัยสำคัญ

Stability ของ Selection Sort

- Selection Sort ไม่ใช่ stable sort
- เหตุผลคือกลไกการ swap ของ Selection Sort อาจ “ดึง” element ที่อยู่ไกลออกไปข้างหลังให้ข้ามหน้า element อื่นที่มีค่าเท่ากันแต่อยู่ใกล้กว่าได้
- ลองพิจารณาตัวอย่าง array [3a, 3b, 1] โดยที่ 3a และ 3b มีค่าเท่ากันคือ 3 แต่เป็นคนละ element กัน (สมมติให้ตัวห้อย a, b แทนตัวตนที่แตกต่างกันเพื่อการติดตาม)
- ใน pass แรก Selection Sort ค้นหาค่าน้อยที่สุดในทั้ง array พบว่าเป็น 1 ที่ตำแหน่ง index 2 จึงทำการสลับตำแหน่ง 0 กับตำแหน่ง 2
  - [3a, 3b, 1] → swap(0, 2) → [1, 3b, 3a]
- สังเกตผลลัพธ์ที่ได้อย่างละเอียด:
  - ก่อนการเรียงลำดับ 3a อยู่ก่อน 3b
  - แต่หลังการเรียงลำดับ 3a กลับไปอยู่**หลัง** 3b แทน ทั้งที่ทั้งคู่มีค่าเท่ากันทุกประการ (คือ 3)
  - ลำดับสัมพัทธ์ระหว่างทั้งสองจึงถูกสลับไปจากเดิม
  - นี่คือเหตุผลที่ Selection Sort ถูกจัดเป็น unstable sort

Insertion Sort

หลักการทำงานของ Insertion sort

- Insertion Sort เปรียบเทียบได้กับการเรียงไพ่ในมือของผู้เล่นไพ่
- ลองจินตนาการว่ากำลังหยิบไพ่ทีละใบจากกองมาเรียงในมือ
  - ไพ่ใบแรกที่หยิบมาย่อมถือว่า “เรียงแล้ว” โดยอัตโนมัติ (เพราะมีอยู่ใบเดียว)
  - เมื่อหยิบไพ่ใบถัดไป จะต้อง “เสียบ” ไพ่ใบนั้นเข้าไปในตำแหน่งที่ถูกต้อง ภายในกลุ่มไพ่ที่เรียงไว้แล้วในมือ
    - โดยเลื่อนไพ่ใบอื่นๆ ที่มีค่ามากกว่าให้ขยับไปทางขวาเพื่อเปิดช่องว่างให้ไพ่ใบใหม่เสียบเข้าไปได้พอดี
- ในเชิง array, algorithm นี้แบ่ง array ออกเป็นสองส่วนเช่นเดียวกับ Selection Sort คือ ส่วนซ้ายที่เรียงแล้วและส่วนขวาที่ยังไม่เรียง แต่กลไกการทำงานแตกต่างกันอย่างสิ้นเชิง
  - Insertion Sort จะหยิบ element ตัวแรกของส่วนขวามาเสมอ (เรียกว่า key) แล้วนำไปเสียบในตำแหน่งที่ถูกต้องของส่วนซ้ายที่เรียงแล้ว

Trace การทำงานบน [5, 3, 8, 1, 2]

- i=1: key = 3
  - ส่วนที่เรียงแล้วคือ [5]
  - เปรียบเทียบ 5 กับ 3 พบว่า 5 > 3 จึงเลื่อน 5 ไปทางขวาหนึ่งช่อง แล้วเสียบ 3 ลงในตำแหน่งที่ว่าง
  - [5, 3, 8, 1, 2] → เลื่อน 5 → เสียบ 3 → [3, 5, 8, 1, 2]
- **i=2:** key = 8
  - ส่วนที่เรียงแล้วคือ [3, 5]
  - เปรียบเทียบ 5 กับ 8 พบว่า 5 < 8 จึงหยุดทันทีโดยไม่ต้องเลื่อนใดๆ เลย แล้วเสียบ 8 ไว้ที่ตำแหน่งเดิมของมัน
  - [3, 5, 8, 1, 2] (ไม่เปลี่ยนแปลง เพราะ 8 อยู่ในตำแหน่งที่ถูกต้องอยู่แล้ว)
- **i=3:** key = 1
  - ส่วนที่เรียงแล้วคือ [3, 5, 8]
  - เปรียบเทียบจากขวาไปซ้าย พบว่า 8, 5, 3 ล้วนมากกว่า 1 ทั้งหมด จึงเลื่อนทั้งสามตัวไปทางขวา แล้วเสียบ 1 ไว้ที่ตำแหน่งแรกสุด
  - เลื่อน 8, 5, 3 → เสียบ 1 → [1, 3, 5, 8, 2]
- **i=4:** key = 2
  - ส่วนที่เรียงแล้วคือ [1, 3, 5, 8]
  - เปรียบเทียบจากขวาไปซ้าย พบว่า 8, 5, 3 มากกว่า 2 (แต่ 1 ไม่มากกว่า) จึงเลื่อนสามตัวนั้นไปทางขวา แล้วเสียบ 2 ไว้ต่อจาก 1
  - เลื่อน 8, 5, 3 → เสียบ 2 → [1, 2, 3, 5, 8]
- **ผลลัพธ์สุดท้าย:** [1, 2, 3, 5, 8] ✓

Best Case O(n)

- จากตัวอย่าง ในขั้นตอน i=2 จะสังเกตเห็นพฤติกรรมที่สำคัญมาก
  - เมื่อ key มีค่ามากกว่า element ตัวสุดท้ายของส่วนที่เรียงแล้ว (คือ 8 มากกว่า 5) การเปรียบเทียบจะหยุดลงทันทีเพียงครั้งเดียว โดยไม่ต้องเลื่อน element ใดๆ เลย
- หาก array ที่ได้รับมาเรียงลำดับไว้อยู่แล้วตั้งแต่ต้น สถานการณ์แบบนี้จะเกิดขึ้นซ้ำๆ ในทุก i เพราะ key แต่ละตัวจะมากกว่า element ก่อนหน้ามันเสมออยู่แล้ว (เนื่องจาก array เรียงแล้ว)
  - การเปรียบเทียบจึงหยุดทันทีหลังจากตรวจสอบเพียงครั้งเดียวในทุกรอบ ทำให้แต่ละ pass ใช้เวลาเพียง O(1)
  - เมื่อรวมทั้งหมด n pass เข้าด้วยกัน จะได้ความซับซ้อนโดยรวมเป็น O(n) ซึ่งเป็น best case ที่ดีที่สุดในบรรดา 3 อัลกอริทึมที่เรียนในสัปดาห์นี้
- ในทางปฏิบัติ คุณสมบัตินี้ทำให้ Insertion Sort เป็น algorithm ที่เร็วที่สุดสำหรับ 2 สถานการณ์เฉพาะ คือ
  - (1) array ที่มีขนาดเล็กมาก (โดยทั่วไปถือว่า n น้อยกว่า 20)
  - (2) array ที่เกือบเรียงลำดับแล้ว (nearly-sorted data) คือมีเพียง element จำนวนน้อยที่อยู่ผิดตำแหน่งเมื่อเทียบกับข้อมูลทั้งหมด

กลไกการ “เลื่อน” แทนการ “สลับ”

- ข้อแตกต่างเชิงเทคนิคที่สำคัญระหว่าง Insertion Sort กับ Bubble Sort คือ วิธีการจัดการกับ element ที่ต้องย้ายตำแหน่ง
  - Bubble Sort ใช้การ swap ซึ่งเป็นการแลกเปลี่ยนตำแหน่งระหว่าง element สองตัว (arr[j], arr[j+1] = arr[j+1], arr[j] ซึ่งภายในเป็นการกระทำสามขั้นตอนคือ เก็บค่าชั่วคราว, เขียนค่าแรก, เขียนค่าที่สอง)
  - Insertion Sort ใช้วิธีที่ต่างออกไป คือ เก็บค่า key ไว้ในตัวแปรชั่วคราวเพียงตัวเดียวก่อน แล้วจึง “เลื่อน” (shift) element ที่มากกว่าไปทางขวาทีละตัว (เขียนทับตำแหน่งเดิมด้วยค่าจาก element ก่อนหน้า) จนกว่าจะพบตำแหน่งที่ถูกต้อง จึงค่อยเสียบ key ลงไปในตอนท้าย
    - การเลื่อนแต่ละครั้งใช้เพียงหนึ่ง operation การเขียน
    - ในขณะที่การ swap หนึ่งครั้งต้องการสองการเขียน (เขียนทับทั้งสองตำแหน่ง)
- ดังนั้นแม้ Bubble Sort และ Insertion Sort จะมีความซับซ้อนแบบ Big-O เดียวกันในกรณีทั่วไป ($O(n^2)$)
  - แต่ Insertion Sort มักใช้จำนวน operation จริงน้อยกว่า Bubble Sort
  - เพราะกลไกการเลื่อนมีต้นทุนต่ำกว่าการสลับ

Stability ของ Insertion Sort

- Insertion Sort เป็น stable sort เช่นเดียวกับ Bubble Sort
- เหตุผลอยู่ที่เงื่อนไขการเลื่อน element ที่ใช้ arr[j] > key (มากกว่าอย่างเคร่งครัด)
- เมื่อพบ element ที่มีค่าเท่ากับ key พอดี เงื่อนไขนี้จะเป็นเท็จทันที ทำให้การเลื่อนหยุดลง และ key จะถูกเสียบไว้หลังจาก element ที่เท่ากันนั้น
    - รักษาลำดับสัมพัทธ์เดิมของ element ที่มีค่าเท่ากันไว้ได้อย่างสมบูรณ์

Code ของทั้ง 3 อัลกอริทึม

Bubble Sort

```python
def bubble_sort(arr: list) -> list:
    """
    Bubble Sort พร้อม early termination
    Best: O(n) | Average: O(n²) | Worst: O(n²) | Space: O(1) | Stable: Yes
    Return: list ใหม่ที่เรียงแล้ว (ไม่แก้ไข input)
    """
    arr = arr.copy()
    n = len(arr)
    comparisons = 0
    swaps = 0
    for i in range(n - 1):
        swapped = False
        for j in range(n - 1 - i):      # ลด range ทุก pass -- ส่วนท้ายเรียงแล้ว
            comparisons += 1
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swaps += 1
                swapped = True
        if not swapped:                  # ไม่มี swap → เรียงแล้ว → หยุด
            break
    print(f"  comparisons={comparisons}, swaps={swaps}")
    return arr
```

- arr = arr.copy() ที่บรรทัดแรกป้องกันไม่ให้ฟังก์ชันแก้ไข list ต้นฉบับที่ผู้เรียกส่งเข้ามา
- ตัวแปร comparisons และ swaps ไม่จำเป็นต่อการทำงานของ algorithm แต่ใส่ไว้เพื่อวัตถุประสงค์ทางการศึกษา ให้เห็นภาพเชิงปริมาณของสิ่งที่เกิดขึ้นจริงเบื้องหลัง
- เงื่อนไข if not swapped: break คือหัวใจของ early termination

ทดสอบ Bubble Sort

```
data = [5, 3, 8, 1, 9, 2]
result = bubble_sort(data)
print(f"  {data} → {result}")

sorted_data = [1, 2, 3, 4, 5]
print("\nBubble Sort (already sorted):")
bubble_sort(sorted_data)
```

```
comparisons=15, swaps=8
[5, 3, 8, 1, 9, 2] → [1, 2, 3, 5, 8, 9]

Bubble Sort (already sorted):
comparisons=4, swaps=0
```

- สังเกตว่า array ที่เรียงลำดับไว้แล้วใช้ comparisons เพียง n - 1 = 4 ครั้ง แล้วหยุดทันที (เพราะ pass แรกไม่มีการ swap เลย)
  - นี่คือ best case O(n) ที่เกิดจาก early termination
- ในทางตรงกันข้าม สำหรับ [5, 3, 8, 1, 9, 2] ซึ่งไม่ได้เรียงมาก่อน
  - early termination ไม่มีโอกาสเกิดขึ้นเลยจนกว่าจะถึง pass สุดท้าย
  - ทำให้ comparisons รวมเท่ากับผลรวมแบบเต็ม 5+4+3+2+1 = 15 พอดี

Selection Sort

```python
def selection_sort(arr: list) -> list:
    """
    Selection Sort — ค้นหา minimum แล้ว swap มาวางหน้า
    Best: O(n²) | Average: O(n²) | Worst: O(n²) | Space: O(1) | Stable: No
    """
    arr = arr.copy()
    n = len(arr)
    comparisons = 0
    swaps = 0
    for i in range(n - 1):
        min_idx = i                          # สมมติว่า minimum อยู่ที่ตำแหน่ง i ก่อน
        for j in range(i + 1, n):
            comparisons += 1
            if arr[j] < arr[min_idx]:
                min_idx = j                  # พบ minimum ตัวใหม่
        if min_idx != i:
            arr[i], arr[min_idx] = arr[min_idx], arr[i]
            swaps += 1
    print(f"  comparisons={comparisons}, swaps={swaps}")
    return arr
```

- ตัวแปร min_idx เริ่มต้นด้วยการสมมติว่า element ที่ตำแหน่ง i เองคือค่าน้อยที่สุดไปก่อน
  - แล้วจึงค้นหาต่อในส่วนที่เหลือว่ามีค่าใดน้อยกว่านี้อีกหรือไม่
- เงื่อนไข if min_idx != i ที่อยู่นอก inner loop ทำหน้าที่ตรวจสอบว่าจำเป็นต้อง swap จริงหรือไม่
  - หาก element ที่ตำแหน่ง i เป็นค่าน้อยที่สุดอยู่แล้วตั้งแต่ต้น (min_idx ยังคงเท่ากับ i) ก็ไม่จำเป็นต้อง swap ให้เสียเวลา
  - นี่คือเหตุผลที่ตัวเลข swaps มักน้อยกว่า n-1 ในทางปฏิบัติ

ทดสอบ Selection Sort

```
data = [5, 3, 8, 1, 9, 2]
result = selection_sort(data)
print(f"  {data} → {result}")

sorted_data = [1, 2, 3, 4, 5]
print("\nSelection Sort (already sorted):")
selection_sort(sorted_data)

data2 = [5, 4, 3, 2, 1]   # worst case ของ Bubble แต่ Selection ก็ยัง swap น้อย
print("\nSelection Sort (reverse sorted):")
result2 = selection_sort(data2)
print(f"  {data2} → {result2}")
```

```
comparisons=15, swaps=4
[5, 3, 8, 1, 9, 2] → [1, 2, 3, 5, 8, 9]

Selection Sort (already sorted):
comparisons=10, swaps=0

Selection Sort (reverse sorted):
comparisons=10, swaps=2
[5, 4, 3, 2, 1] → [1, 2, 3, 4, 5]
```

- ค่า comparisons = 10 เท่ากันทุกกรณี สำหรับ n=5 (คำนวณจาก 4+3+2+1 = 10)
  - ไม่ว่า input จะเรียงมาแล้ว เรียงกลับหัว หรือแบบใดก็ตาม
- Selection Sort เป็น $O(n^2)$ ทุกกรณีโดยไม่มีข้อยกเว้น
- ในขณะที่ค่า swaps กลับแตกต่างกันไปตามลักษณะของ input
  - 0 สำหรับเรียงแล้ว
  - 2 สำหรับเรียงกลับหัว
- แสดงให้เห็นว่า comparisons ไม่ขึ้นกับ input แต่ swaps ขึ้นกับ input

Insertion Sort

```python
def insertion_sort(arr: list) -> list:
    """
    Insertion Sort — เสียบ element ใหม่เข้าตำแหน่งที่ถูกต้องในส่วนที่เรียงแล้ว
    Best: O(n) | Average: O(n²) | Worst: O(n²) | Space: O(1) | Stable: Yes
    """
    arr = arr.copy()
    comparisons = 0
    shifts = 0          # นับการ "เลื่อน" ไม่ใช่ "swap"
    for i in range(1, len(arr)):
        key = arr[i]                            # element ที่จะนำไปเสียบ
        j = i - 1                               # เริ่มตรวจจาก element ซ้ายสุดของ key
        while j >= 0 and arr[j] > key:    # ตรวจ boundary ก่อนเสมอ (short-circuit)
            comparisons += 1
            arr[j + 1] = arr[j]                 # เลื่อนไปขวาหนึ่งช่อง
            shifts += 1
            j -= 1
        if j >= 0:                          # นับ comparison ครั้งสุดท้ายที่ทำให้ while หยุด
            comparisons += 1              # (กรณีหยุดเพราะ arr[j] <= key ไม่ใช่เพราะ j<0)
        arr[j + 1] = key                        # เสียบ key ลงในตำแหน่งที่ถูกต้อง
    print(f"  comparisons={comparisons}, shifts={shifts}")
    return arr
```

- เงื่อนไข while j >= 0 and arr[j] > key ใช้ประโยชน์จาก short-circuit evaluation ของ Python
  - เมื่อ j >= 0 เป็น False แล้ว Python จะไม่ประเมินเงื่อนไข arr[j] > key ต่อเลย ทำให้ปลอดภัยจากการเข้าถึง arr[-1] ซึ่งจะเกิดขึ้นหากตรวจสอบ arr[j] > key ก่อนโดยไม่เช็ค j >= 0 ก่อน

ทดสอบ Insertion Sort

```
data = [5, 3, 8, 1, 9, 2]
result = insertion_sort(data)
print(f"  {data} → {result}")

sorted_data = [1, 2, 3, 4, 5]
print("\nInsertion Sort (already sorted):")
insertion_sort(sorted_data)

nearly = [1, 2, 4, 3, 5]    # สลับแค่คู่เดียว
print("\nInsertion Sort (nearly sorted):")
insertion_sort(nearly)
```

```
comparisons=11, shifts=8
[5, 3, 8, 1, 9, 2] → [1, 2, 3, 5, 8, 9]

Insertion Sort (already sorted):
comparisons=4, shifts=0

Insertion Sort (nearly sorted):
comparisons=5, shifts=1
```

- ผลลัพธ์บน “nearly sorted” ([1, 2, 4, 3, 5] ซึ่งมีเพียงคู่ 4 กับ 3 ที่สลับกัน) แสดงให้เห็นชัดเจนว่าใช้ comparisons เพียง 5 ครั้งและ shifts เพียง 1 ครั้งเท่านั้น ใกล้เคียงกับกรณี already-sorted มาก (4 comparisons, 0 shifts)
- นี่คือหลักฐานที่เป็นรูปธรรมว่า Insertion Sort มีประสิทธิภาพโดดเด่นเป็นพิเศษบน nearly-sorted data

ตาราง Complexity เปรียบเทียบ

Algorithm	Best	Average	Worst	Space	Stable	จุดแข็ง
Bubble Sort (optimized)	O(n)	O(n²)	O(n²)	O(1)	✓	เข้าใจง่าย, O(n) บน sorted data
Selection Sort	O(n²)	O(n²)	O(n²)	O(1)	X	swap น้อยสุด (≤ n-1 ครั้ง)
Insertion Sort	O(n)	O(n²)	O(n²)	O(1)	✓	เร็วมากบน nearly-sorted, ดีสำหรับ n เล็ก

เมื่อไหร่ควรเลือกอะไร

- Bubble Sort เหมาะสำหรับการสอนและทำความเข้าใจ concept การจัดเรียงข้อมูลเป็นหลัก
  - ไม่ค่อยถูกนำไปใช้จริงใน production code
  - เพราะแม้จะมี best case ที่ดีบน sorted data แต่โดยรวมแล้วมักช้ากว่า Insertion Sort ในสถานการณ์ใกล้เคียงกัน
- Selection Sort เหมาะเมื่อ write operation มีต้นทุนสูงมากเป็นพิเศษ
  - เช่นหน่วยความจำประเภท EEPROM หรือ Flash memory ที่การเขียนแต่ละครั้งก่อให้เกิดการสึกหรอของฮาร์ดแวร์
- Insertion Sort เหมาะเมื่อข้อมูลเกือบเรียงลำดับแล้ว เมื่อ n มีขนาดเล็กกว่า 20 หรือถูกใช้เป็น sub-routine ภายใน algorithm ที่ซับซ้อนกว่า เช่น Merge Sort และ Timsort
  - algorithm การเรียงลำดับที่ Python ใช้จริงในฟังก์ชัน sorted() และ list.sort() ใช้ Insertion Sort จัดการ partition ขนาดเล็กภายในตัวเองด้วย

Stability คืออะไรและทำไมสำคัญ

- Sort algorithm ถูกจัดว่าเป็น stable เมื่อ element ที่มีค่าเท่ากันยังคงอยู่ในลำดับสัมพัทธ์เดิมหลังการเรียงลำดับเสร็จสิ้น
- คุณสมบัตินี้สำคัญมากในสถานการณ์ที่ต้องเรียงลำดับข้อมูลหลายระดับต่อเนื่องกัน (multi-key sorting)
  - เช่น ต้องการเรียงรายชื่อนิสิตตาม GPA เป็นหลัก และในกรณีที่ GPA เท่ากันให้เรียงตามชื่อเป็นตัวตัดสิน
  - วิธีที่ถูกต้องคือเรียงตามชื่อก่อน แล้วจึงเรียงตาม GPA ทับอีกครั้งด้วย stable sort
  - เพราะ stable sort จะไม่ทำลายลำดับตามชื่อที่เรียงไว้ก่อนหน้าสำหรับกลุ่มที่มี GPA เท่ากัน
  - แต่หาก sort ที่ใช้ในขั้นตอนที่สองเป็น unstable ลำดับชื่อภายในกลุ่ม GPA เดียวกันอาจถูกสลับไปโดยไม่มีเหตุผลรองรับ ทำให้ผลลัพธ์สุดท้ายดูสับสนไม่เป็นระบบ